

UNITAT 8 · SETMANES 13–14

Xarxa: els sòcols

📅 Setmanes 13–14 · dates per confirmar </> Laboratori TD7 (resolt + repte IA)

~ 2 animacions

LABORATORI TD7 · RESOLT

Xarxa amb sòcols: d'un servidor mínim a un servidor web amb cinc models

Un sòcol es fa servir com un fitxer (`read`, `write`, `close`), però connecta processos en màquines diferents. Aquest laboratori ho ajunta tot: `fork` per client, un fil per connexió i el productor–consumidor del TD6 per atendre molts clients alhora.

EXERCICIS

- | | | | |
|---|--|---|--|
| 1 | El mínim imprescindible d'un servidor | 4 | Exploració: estats TCP, la cua de |
| 2 | Servidor d'eco, iteratiu i amb <code>fork</code> | | <code>listen</code> , els models amb càrrega i |
| 3 | Servidor web: iteratiu, un fil per connexió i grup | | UDP |

TD7 · 1

● Resolt · provat

El mínim imprescindible d'un servidor

Enunciat. Un programa que espera una connexió (prova'l amb `telnet` o `nc`) i acaba: `socket` (IPv4, orientat a connexió) → `sockaddr_in` amb `INADDR_ANY` i el port en ordre de xarxa (`htons`) → `bind` → `listen` → `accept` → `close`.

Solució

C tds/td7/sock.c

```
/*
 * TD7 - exercise 1: the server side of a TCP connection, step by step.
 * ./sock [port]      (default 7000)  then, from another terminal:
telnet localhost 7000
 * socket → bind → listen → accept → close.
 */
```

```

#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000

int main(int argc, char *argv[])
{
    int port = argc > 1 ? atoi(argv[1]) : PORT;

    /* 1. An IPv4 (AF_INET), connected, reliable byte stream (SOCK_STREAM =
    TCP). */
    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");

    /* Allow restarting the server right away (otherwise bind fails for ~60
    s: TIME_WAIT). */
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
    "setsockopt");

    /* 2. Local address: any interface of this machine, the chosen port
    (network byte order). */
    struct sockaddr_in addr = { 0 };
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);

    /* 3. and 4. Bind the address, start listening (queue of 1 pending
    connection). */
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 1) == -1, "listen");
    printf("listening on port %d\n", port);
    fflush(stdout);

    /* 5. accept blocks until a client connects and returns a NEW descriptor
    for it. */
    int client = accept(s, NULL, NULL);
    exit_if(client == -1, "accept");
    printf("connection accepted, closing\n");
  
```

```
/* 6. Close the connection and the listening socket. */
exit_if(close(client) == -1 || close(s) == -1, "close");
return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./sock 37634 & sleep 0.3; nc -z 127.0.0.1 37634 && echo 'client connected';
wait
listening on port 37634
connection accepted, closing
Connection to 127.0.0.1 37634 port [tcp/*] succeeded!
client connected
```

- `accept` retorna un **descriptor nou** per a aquest client; el sòcol d'escolta queda disponible per acceptar-ne d'altres.
- `htons` / `htonl`: la xarxa fa servir l'ordre *big-endian*; x86 és *little-endian* (ho vas veure al TD1 amb `od`).
- `SO_REUSEADDR` permet tornar a engegar el servidor de seguida: sense ell, `bind` falla amb `EADDRINUSE` mentre les connexions anteriors són en `TIME_WAIT`.

Animació A8.1 · Cicle de vida d'una connexió →

TD7 · 2

● Result · provat

Servidor d'eco, iteratiu i amb `fork`

Enunciat. Un servidor d'eco retorna al client tot el que rep, excepte `quit`, que tanca la connexió i n'espera una altra. Llança diversos clients alhora: què passa? Canvia el servidor amb `fork` perquè n'atengui diversos.

Solució

C tds/td7/echo_server.c

```
/*
 * TD7 - exercise 2: echo server. Sends back every line it receives; "quit"
 * closes the
 * connection and the server waits for the next client.
 */
```

```

*  ./echo_server [port] [-f]      -f: one child process per client (fork)
*  Without -f the server is ITERATIVE: a second telnet waits until the first
one leaves
*  (its connection stays in the listen queue). With -f each client gets its
own process.
*/
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#include "utils.h"

#define PORT 7000
#define LINE 1024

/* Echo line by line until end of file or "quit". Returns when the client is
done. */
static void serve(int c)
{
    char line[LINE];
    size_t len = 0;

    for (;;) {
        ssize_t n = read(c, line + len, 1);      /* byte by byte: simple,
fine for a lab */
        if (n == -1 && errno == EINTR)
            continue;
        if (n <= 0)                               /* 0: the client
closed; -1: error */
            return;
        if (line[len] == '\n' || len == LINE - 2) {
            len++;
            if (strncmp(line, "quit", 4) == 0)
                return;
            if (write_all(c, line, len) == -1)
                return;
            len = 0;
        } else {
            len++;
        }
    }
}

```

```

    }
}

int main(int argc, char *argv[])
{
    int port = PORT, use_fork = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-f") == 0) use_fork = 1;
        else port = atoi(argv[i]);
    }

    int s = socket(AF_INET, SOCK_STREAM, 0);
    exit_if(s == -1, "socket");
    int yes = 1;
    exit_if(setsockopt(s, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes) == -1,
"setsockopt");
    struct sockaddr_in addr = { .sin_family = AF_INET, .sin_port =
htons((uint16_t)port),
                                .sin_addr.s_addr = htonl(INADDR_ANY) };
    exit_if(bind(s, (struct sockaddr *)&addr, sizeof addr) == -1, "bind");
    exit_if(listen(s, 8) == -1, "listen");
    printf("echo server on port %d (%s)\n", port, use_fork ? "one process per
client" : "iterative");
    fflush(stdout);

    /* Children that finish must not stay zombies: ignoring SIGCHLD makes the
kernel reap them. */
    if (use_fork)
        exit_if(signal(SIGCHLD, SIG_IGN) == SIG_ERR, "signal");

    for (;;) {
        int c = accept(s, NULL, NULL);
        if (c == -1 && errno == EINTR)
            continue;
        exit_if(c == -1, "accept");

        if (!use_fork) {
            serve(c);
            exit_if(close(c) == -1, "close");
            continue;
        }
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0) {
            exit_if(close(s) == -1, "close");           /* the child does not

```

```
accept */
    serve(c);
    _exit(EXIT_SUCCESS);
}
exit_if(close(c) == -1, "close");      /* the parent does not
talk to the client */
}
```

Sortida real

```
$ python3 tcp_client.py echo 22751 'hola\nmundo\nquit'
sent: hola      received: hola
sent: mundo     received: mundo
```

Sortida real

```
$ python3 tcp_client.py blocked 22751      # iterative server
client B got nothing after 1 s: the server is busy with client A
```

Sortida real

```
$ python3 tcp_client.py blocked 34885      # server with fork
client B got 'hi\n' after 0.00 s
```

El servidor iteratiu atén un client cada vegada: mentre el client A està connectat, B espera a la cua de `listen` i no rep resposta (la prova ho mesura: res en 1 s). Amb `-f`, cada connexió té el seu procés i B rep el seu eco de seguida. El pare tanca el sòcol del client i el fill el d'escolta; ignorar `SIGCHLD` evita els zombis. La versió heretada detectava `quit` amb `strstr` sobre una memòria intermèdia sense terminator; ara llegeix línia a línia.

Servidor web: iteratiu, un fil per connexió i un grup de 8 fils

Enunciat. El client (un navegador) envia `GET fitxer`; el servidor respon `HTTP/1.0 200 OK`, el tipus MIME i el contingut. Fes servir `stat` per saber si existeix i quina mida té, `mmap` per mapar-lo i `write` per enviar-lo. Després: (1) un fil nou per connexió; (2) 8 fils creats a l'inici que esperen connexions —el servidor *produeix* connexions i els fils les *consumeixen*.

Solució



C tds/td7/http_server.c

```

/* Handles one request on the connected socket c. Never exits the process. */
static void handle(int c)
{
    char req[REQUEST], path[1024], full[2048];
    ssize_t n = read(c, req, sizeof req - 1);
    if (n ≤ 0)
        return;
    req[n] = '\0';

    if (sscanf(req, "GET %1023s", path) ≠ 1) {
        reply_error(c, "400 Bad Request");
        return;
    }
    if (strstr(path, "..") ≠ NULL) {                                /* no escape from root */
        reply_error(c, "403 Forbidden");
        return;
    }
    if (strcmp(path, "/") = 0)
        strcpy(path, "/index.html");
    snprintf(full, sizeof full, "%s%s", root, path);

    struct stat st;
    if (stat(full, &st) = -1 || !S_ISREG(st.st_mode)) {
        reply_error(c, "404 Not Found");                            /* PG204 answered
"201" by mistake */
        return;
    }
    int fd = open(full, O_RDONLY);
    if (fd = -1) {
        reply_error(c, "403 Forbidden");
        return;
    }
    char head[256];
    int hn = snprintf(head, sizeof head,
                      "HTTP/1.0 200 OK\r\nContent-Type: %s\r\nContent-Length:
%lld\r\n\r\n",
                      mime_type(full), (long long)st.st_size);
    if (write_all(c, head, (size_t)hn) = -1) {
        close(fd);
    }

```

```

    return;
  }
  if (st.st_size > 0) {
    /* mmap of 0 bytes is
    an error */
    void *data = mmap(NULL, (size_t)st.st_size, PROT_READ, MAP_PRIVATE,
    fd, 0);
    if (data != MAP_FAILED) {
      (void)write_all(c, data, (size_t)st.st_size); /* loops over
      partial writes */
      munmap(data, (size_t)st.st_size);
    }
  }
  close(fd);
}

/* ----- thread per connection ----- */
static void *connection_thread(void *arg)
{
  int c = (int)(intptr_t)arg;
  handle(c);
  close(c);
  return NULL;
}

/* ----- pool: bounded queue of connected sockets ----- */
static int queue[QUEUE];
static int q_in = 0, q_out = 0;
static sem_t q_free, q_used;
static pthread_mutex_t q_lock = PTHREAD_MUTEX_INITIALIZER;

static void *worker(void *arg)
{
  (void)arg;
  for (;;) {
    sem_wait(&q_used); /* wait for a connection */
    pthread_mutex_lock(&q_lock);
    int c = queue[q_out];
    q_out = (q_out + 1) % QUEUE;
    pthread_mutex_unlock(&q_lock);
    sem_post(&q_free);
    handle(c);
    close(c);
  }
  return NULL;
}

```

Sortida real

```
$ curl -s -i http://localhost:PORT/ | tr -d '\r' # mode pool
HTTP/1.0 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 20

<h1>FS0 · UTI</h1>
```

Sortida real

```
$ curl -s -o /dev/null -w '%{http_code}' ../missing.html ; curl --path-as-is
.../..etc/passwd
404
403
```

MODE	COM ATÉN	LÍMIT
<code>-m iterative</code>	Un client cada vegada	Un client lent bloqueja tothom
<code>-m fork</code>	Un procés fill per connexió	Crear un procés costa diverses vegades més que un fil (mesurat al TD6 · 5)
<code>-m thread</code>	Un fil nou per connexió (<code>pthread_detach</code>)	Milers de connexions = milers de fils (memòria, canvis de context)
<code>-m pool</code>	8 fils fixos + cua limitada (semàfors + mutex)	Recursos limitats; 8 clients lents el saturen (TD7 · 4)
<code>-m epoll</code>	Un fil i un bucle d'esdeveniments: només atén els sòcols que ja tenen dades	Aquesta versió senzilla encara escriu de manera bloquejant; nginx o Node.js també escriuen sense bloquejar

Les proves envien 40 peticions concurrents (8 alhora) en cada mode i comproven 40 respostes 200 i un binari idèntic a l'original. Seguretat mínima: un camí amb `..` rep **403** (el client no pot sortir de la carpeta servida) i el que no existeix, **404**. El servidor ignora SIGPIPE perquè no mori si un client es desconnecta enmig de la resposta.



Corregit i completat

La versió heretada responia `201 Not found` (el codi correcte és 404), no bloquejava `..`, no enviava `Content-Length`, ignorava les escriptures parcials, no comprovava l'obertura del fitxer i no tenia les versions amb fils i amb grup que demana l'enunciat.

[Animació A8.2 · Models de servidor →](#)

TD7 · 4

● Nou · mesurat a la màquina del curs

Exploració: què fa el nucli amb els teus sòcols

Enunciat. (1) Llegeix amb `getsockopt(TCP_INF0)` l'estat de cada sòcol a cada pas d'una connexió. (2) Descobreix què limita el backlog de `listen`. (3) Mesura els cinc modes del servidor web amb clients lents. (4) Compara TCP i UDP. (5) Comprova l'error d' `htons` de les dispositives originals i resol noms amb `getaddrinfo`. (6) Crea un sòcol local.

1 · Estats TCP (`tcp_states.c`)



C tds/td7/tcp_states.c

```

    struct tcp_info ti;
    socklen_t len = sizeof ti;
    exit_if(getsockopt(s, IPPROTO_TCP, TCP_INFO, &ti, &len) == -1,
"getsockopt");
    return ti;
}

static const char *state(int s)
{
    if (s == -1)
        return "-";
    unsigned st = info(s).tcpi_state;
    return st < sizeof NAMES / sizeof NAMES[0] ? NAMES[st] : "?";
}

/* One line per step: the state of each socket. For a listening socket the
kernel reports the
 * accept queue in tcpi_unacked (connections waiting for accept) and
tcpi_sacked (backlog). */
static void show(const char *step, int l, int c, int a)
{
    struct tcp_info li = info(l);
    printf("%-26s | listener %s (queue %u/%u) | client %-11s | server %s\n",
        step, state(l), li.tcpi_unacked, li.tcpi_sacked, state(c),
state(a));
}

```

Sortida real

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -
| server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED
| server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED
| server ESTABLISHED
write/read                 | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2
| server CLOSE_WAIT
server read                | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED
| server CLOSED
                           | the client's end moved to a TIME_WAIT entry of
the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

Tres fets que mostra la sortida: la connexió està **establerta abans d' `accept`** (el nucli fa l'encaixada; la cua passa a 1/4); qui tanca primer passa per `FIN_WAIT2` mentre l'altre queda en `CLOSE_WAIT` fins que el seu `read` retorna 0; i després del tancament el nucli guarda una entrada **`TIME_WAIT`** (60 s) que `ss` mostra encara que el programa hagi acabat. Per això els servidors fan servir `SO_REUSEADDR`.

2 · Què limita el backlog (`backlog.c`)

Sortida real

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5,
SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8,
SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

Amb `listen(fd, 4)` Linux completa **`backlog + 1 = 5`** encaixades i descarta el SYN dels altres, que queden en `SYN_SENT` i el **tornen a enviar** (1 s, després 2 s, 4 s...). Quan el servidor accepta, la cua es buida i els reenviaments entren. El backlog no limita

quants clients atén el servidor: limita quantes connexions acabades poden esperar `accept`.



Per als curiosos: les SYN cookies

Sortida real

```
$ nstat -n; ./backlog 2 8; nstat | grep -E
'Syncookies|ListenOverflows'
listen(2), 8 clients connect
0.3 s, no accept yet:      accept queue 3 | clients
ESTABLISHED 3, SYN_SENT 5
accept 3 (total 3), wait:  accept queue 3 | clients
ESTABLISHED 8, SYN_SENT 0
accept 3 (total 6), wait:  accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
accept 0 (total 6), wait:  accept queue 0 | clients
ESTABLISHED 8, SYN_SENT 0
TcpExtSyncookiesSent 2
TcpExtListenOverflows 7
```

Amb `listen(fd, 2)` passa una cosa més subtil: quan també s'omple la cua d'encaixades a mig fer, el nucli respon amb **SYN cookies** (sense guardar estat, una defensa contra les inundacions de SYN). Si la cua d'acceptació és plena quan arriba l'ACK final, aquest ACK es perd: el client es creu ESTABLISHED, però el servidor no en sap res. Els comptadors del nucli (`nstat`) ho confirmen. Per això el backlog d'un servidor real és gran (en aquesta màquina, `net.core.somaxconn` = 4096).

3 · Els cinc models amb càrrega (`load.c`)



32 clients: primer 1 (i després 8) que triguen 1 s a enviar la petició, com un client en una xarxa dolenta o un atac *slowloris*; 20 ms més tard, els ràpids. Es mesura la latència de cada grup, des de `connect` fins a la resposta sencera.

Sortida real

```
$ ./http_server -m iterative & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 981 ms, max 982 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 982 ms, max 983 ms
```

Sortida real

```
$ ./http_server -m pool & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 1 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 981 ms, max 981 ms
```

Sortida real

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load PORT 8 24 1000
32 replies 200 OK of 32
slow 1: median 1001 ms, max 1001 ms
fast 31: median 1 ms, max 2 ms
32 replies 200 OK of 32
slow 8: median 1001 ms, max 1001 ms
fast 24: median 1 ms, max 2 ms
```

Iteratiu: un sol client lent fa esperar tothom ~1 s. Grup de 8: aguanta 1 client lent, però amb 8 els vuit fils estan bloquejats a `read` i torna a passar el mateix. `fork`, un fil per client i `epoll` responen els ràpids en pocs mil·lisegons (les sortides `fork` i `thread` són a `tests/out/td7/`). El bucle d'esdeveniments ho fa amb **un sol fil**:

C tds/td7/http_server.c

```

/* ----- event loop: one thread watches every socket ----- */
static void event_loop(int s)
{
    int ep = epoll_create1(0);
    exit_if(ep == -1, "epoll_create1");
    struct epoll_event ev = { .events = EPOLLIN, .data.fd = s }, ready[64];
    exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, s, &ev) == -1, "epoll_ctl");
    for (;;) {
        int n = epoll_wait(ep, ready, 64, -1); /* sleeps until some socket
is readable */
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "epoll_wait");
        for (int i = 0; i < n; i++) {
            int fd = ready[i].data.fd;
            if (fd == s) { /* a new connection: watch
it too */
                int c = accept(s, NULL, NULL);
                if (c == -1)
                    continue;
                ev.data.fd = c;
                exit_if(epoll_ctl(ep, EPOLL_CTL_ADD, c, &ev) == -1,
"epoll_ctl");
            } else { /* the request has arrived:
read won't wait */
                exit_if(epoll_ctl(ep, EPOLL_CTL_DEL, fd, NULL) == -1,
"epoll_ctl");
                handle(fd);
                close(fd);
            }
        }
    }
}

```

Animació A8.2 · Models de servidor →

4 · TCP contra UDP (`udp.c`)



Sortida real

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3
"two", recvfrom #3: 5 "three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8
received, 9992 lost
      kernel counter Udp RcvbufErrors grew by 9992
```

TCP és un **flux de bytes**: tres `write` arriben en un sol `read` (com amb una canonada), així que el protocol d'aplicació ha de marcar on acaba cada missatge. UDP conserva els **límits** de cada datagrama, però no espera el receptor: amb la memòria intermèdia plena, el nucli descarta datagrames sense avisar ningú, fins i tot dins de la mateixa màquina, i només ho anota a `/proc/net/snmp`.

5 · Adreces i ports (`addr.c`) · 6 · Sòcol local (`local.c`)

Sortida real

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:          the kernel bound port 22555
sin_port = htons(7000):   the kernel bound port 7000
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
ip_unprivileged_port_start = 1024
sin_port = htons(80):     bind: Permission denied
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28, sockaddr_un 110,
sockaddr_storage 128
```

Sortida real

```
$ ./local fso.sock
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path
holds 108 bytes
the server asks the kernel who connected: pid = mine, uid 1000 (my user)
read 5 bytes; after close the file stays until unlink
still there after close: yes
```

L'exemple de la diapositiva 187 posava `sin_port = 7000` sense `htons`: en un processador *little-endian* els dos bytes es llegeixen al revés i el nucli obre el port **22555**. `gethostbyname` (també en aquella diapositiva) només coneix IPv4, no és segur amb fils i va sortir de POSIX el 2008: `getaddrinfo` resol IPv4 i IPv6 amb la mateixa crida. Un sòcol `AF_UNIX` no fa servir 127.0.0.1 (això és `AF_INET` sobre la interfície de bucle local): la seva adreça és un **fitxer de tipus sòcol**, i el nucli diu al servidor qui s'ha connectat (`SO_PEERCRED`), cosa que fan servir systemd, D-Bus o Docker per decidir permisos.

Animació A8.1 · Cicle de vida d'una connexió →

Llista de comprovació

Repte amb IA generativa · Un servidor `epoll` que no es bloqueja mai



Objectiu. Converteix el mode `-m epoll` d'`http_server.c` (que encara llegeix i escriu de manera bloquejant) en un bucle d'esdeveniments complet, amb sòcols no bloquejants i escriptures parcials, i mesura'l contra el mode d'un fil per client amb milers de connexions.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

Tinc un servidor HTTP en C amb cinc modes; el mode `epoll` fa servir un sol fil, però `handle()` fa un `read` i un `write` que es poden bloquejar [enganxa `http_server.c` i `load.c`]. Ajuda'm a: (1) fer servir `accept4(..., SOCK_NONBLOCK)` i guardar un estat per connexió (llegir la petició fins a `\r\n\r\n`, escriure la resposta amb un desplaçament), registrant `EPOLLOUT` només quan `write` retorni `EAGAIN`; (2) ampliar `load.c` perquè obri 2000 connexions que envien molt a poc a poc; (3) comparar el mode `epoll` amb `-m thread` mesurant la latència dels clients ràpids, el nombre de fils (`ps -o nlwp`) i la memòria (`ps -o rss`). Explica'm el problema C10K i quin recurs del sistema limita cada mode.

Com comprovar el resultat

- Un client que llegeix un fitxer gran molt a poc a poc (`curl --limit-rate 10k`) no endarrereix els altres: `./load` ho mostra.
- `tests/run.sh td7` continua passant i el fitxer binari arriba idèntic (`cmp`).
- La taula surt de les teves mesures i diu en quina màquina s'han fet.
- `ulimit -n` explica almenys un error que observes en augmentar la concurrència.

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.